

NumPy

Complete Revision Notes

Kabir Roy · 23BCE10815

VIT Bhopal University · B.Tech CSE

Foundations of Data Science

Topics: Speed & Memory · Array Creation · Indexing & Slicing · Multidimensional Arrays ·
Broadcasting · Data Types · Built-in Methods · Data Science Extras

Contents

1	Why NumPy? Speed, Memory & Vectorization	3
1.1	Speed Comparison	3
1.2	Memory Efficiency	3
1.3	Vectorization — No More Loops!	4
1.4	zip() — Quick Reminder	4
2	Creating NumPy Arrays	4
2.1	From a Python List	4
2.2	Built-in Array Creators	5
2.3	Array Properties	5
2.4	Changing Data Type — <code>astype()</code>	6
2.5	Reshape & Flatten	6
3	Indexing & Slicing	6
3.1	Basic Indexing & Slicing (1D)	6
3.2	View vs Copy — CRITICAL!	7
3.3	Fancy Indexing	7
3.4	Boolean Masking	7
3.5	Random Array Generation	8
4	Multidimensional Indexing & Axis	8
4.1	2D Indexing	8
4.2	Axis Concept	8
4.3	3D Array Indexing	9
5	Broadcasting	9
5.1	Scalar Broadcasting	9
5.2	Array Broadcasting	10
5.3	Real-World: Data Normalization	10

6	NumPy Data Types	10
6.1	Integer & Float Types	11
6.2	Other Data Types	11
6.3	Data Type Quick Reference	11
7	Built-in Methods — Quick Reference	12
8	Important NumPy for Data Science (Extra)	12
8.1	Stacking & Concatenation	12
8.2	Sorting	13
8.3	Linear Algebra (numpy.linalg)	13
8.4	where() — Conditional Selection	14
8.5	clip() — Bounding Values	14
8.6	np.nan and Handling Missing Data	14
8.7	np.random with Seeds (Reproducibility)	15
8.8	Splitting Arrays	15
8.9	Useful Utility Functions	15
8.10	Vectorized Math on Arrays	16
	Quick Cheatsheet	16

1 Why NumPy? Speed, Memory & Vectorization

1.1 Speed Comparison

NumPy arrays are **much faster** than Python lists because they use compiled C code internally.

```
1 import numpy as np
2 import time
3
4 size = 1_000_000
5
6 # Python Lists
7 l1 = list(range(size))
8 l2 = list(range(size))
9
10 start = time.time()
11 add = [x + y for x, y in zip(l1, l2)]    # Loop-based
12 end = time.time()
13 print("Python Lists take", end - start)  # ~0.175 sec
14
15 # NumPy Arrays
16 l1 = np.array(range(size))
17 l2 = np.array(range(size))
18
19 start = time.time()
20 add = l1 + l2                          # Vectorized
21 end = time.time()
22 print("NumPy Arrays take", end - start)  # ~0.011 sec
```

★ Note

NumPy was **15x faster** here. The gap grows with larger data.

1.2 Memory Efficiency

```
1 import sys
2 import numpy as np
3
4 list_data = list(range(1000))
5 numpy_data = np.array(list_data)
6
7 print("Python list size:", sys.getsizeof(list_data) *
8       len(list_data), "bytes")
9 # ~8,056,000 bytes
10 print("NumPy array size:", numpy_data.nbytes, "bytes")
```

```

11 # 8,000 bytes (1000 elements * 8 bytes each)
12 # nbytes = Number of elements x bytes per element

```

✓ Tip

Why so different? Python lists store *pointers* to objects (8 bytes each) plus each object's overhead. NumPy stores raw numbers in a *contiguous* block — no overhead.

1.3 Vectorization — No More Loops!

```

1 # Python List way (slow)
2 list1 = [1, 2, 3]
3 list_squares = [x**2 for x in list1]
4 print(list_squares)    # [1, 4, 9]
5
6 # NumPy way (fast + clean)
7 arr1 = np.array([1, 2, 3])
8 numpy_squares = arr1 ** 2
9 print(numpy_squares)   # [1 4 9]

```

✓ Tip

Vectorization uses **SIMD** (Single Instruction, Multiple Data) — the CPU processes multiple elements simultaneously in compiled C code.

1.4 zip() — Quick Reminder

```

1 l1 = [1, 2, 4]
2 l2 = [6, 7, 8]
3 list(zip(l1, l2))    # [(1, 6), (2, 7), (4, 8)]
4 # zip() pairs elements by position as tuples

```

2 Creating NumPy Arrays

2.1 From a Python List

```

1 arr1 = np.array([1, 2, 3])                # 1D array
2 arr2 = np.array([[1,2,3],[3,4,5],[6,6,7]]) # 2D array
3                                           (nested list)
4 # IMPORTANT: Array should ideally contain only numbers.

```

```

5 # Mixing types causes NumPy to upcast everything:
6 np.array([1, 2, 3, 4, 5, "Kabir"])
7 # Output: array(['1', '2', '3', '4', '5', 'Kabir'],
      dtype='<U21') <- all become strings!

```

△ Watch Out!

`np.array()` takes **only one argument** (a list or nested list). You cannot pass multiple separate lists like `np.array([1,2], [3,4])`.

2.2 Built-in Array Creators

```

1 np.zeros((3, 3))           # 3x3 array filled with 0.0
2 np.ones((3, 5))           # 3x5 array filled with 1.0
3 np.full((4, 6), 5)        # 4x6 array filled with 5 --
      (shape, value)
4 np.eye(5)                 # 5x5 Identity matrix (1s on
      diagonal, 0s elsewhere)
5
6 np.arange(1, 100, 9)       # Like range(): start=1, stop=100,
      step=9
7                               # Output: [1, 10, 19, 28, ... 91]
8
9 np.linspace(1, 100, 5)    # 5 equally spaced values from 1 to
      100 (inclusive)
10                               # Output: [1., 25.75, 50.5, 75.25,
      100.]

```

★ Note

arange vs linspace:

- `arange(start, stop, step)` — specify the *step size*
- `linspace(start, stop, n)` — specify the *number of points*

2.3 Array Properties

```

1 myarr = np.array([[1,2,3],[4,5,6],[7,8,9],[10,11,12]])
2
3 print(type(myarr))        # <class 'numpy.ndarray'>
4 myarr.shape               # (4, 3) -- rows x columns
5 myarr.size                # 12 -- total elements
6 myarr.ndim                # 2 -- number of dimensions
7 myarr.dtype               # dtype('int64') -- default int type
8 myarr.nbytes              # total bytes used in memory

```

2.4 Changing Data Type — `astype()`

```

1 myarr = np.array([[1,2,3],[4,5,6]], dtype='float32')    #
    set at creation
2 n = myarr.astype('float64')                            #
    convert later
3 print(n.dtype)    # dtype('float64')
```

2.5 Reshape & Flatten

```

1 myarr = np.array([[1,2,3],[4,5,6],[7,8,9],[10,11,12]])
2 # shape is (4, 3) -- total 12 elements
3
4 myarr.reshape((3, 4))    # New shape must have same total
    elements (3*4=12)
5 # Output: [[ 1  2  3  4], [ 5  6  7  8], [ 9 10 11 12]]
6
7 myarr.flatten()          # Convert to 1D array
8 # Output: [ 1  2  3  4  5  6  7  8  9 10 11 12]
```

★ Note

`reshape()` creates a *view* (shared memory). `flatten()` creates a *copy*.

3 Indexing & Slicing

3.1 Basic Indexing & Slicing (1D)

```

1 arr = np.array([[1,2,3],[4,5,6],[7,8,9],[11,12,13]])
2 flat = arr.flatten()    # [1,2,3,4,5,6,7,8,9,11,12,13]
3
4 flat[0]    # 1    -- element at index 0
5 flat[3]    # 4    -- element at index 3
6 flat[3:6]  # [4, 5, 6] -- slicing (start:stop, stop
    excluded)
7 flat[:5]   # [1, 2, 3, 4, 5] -- from start to index 4
8 flat[3:13] # [4, 5, 6, 7, 8, 9, 11, 12, 13]
9 flat[::2]  # [1, 3, 5, 7, 9, 11, 13] -- every 2nd element
```

3.2 View vs Copy — CRITICAL!

```
1 # SLICING RETURNS A VIEW (not a copy!)
2 b = flat[3:7]      # b = [4, 5, 6, 7]
3 b[0] = 444444      # modifying b...
4 print(flat)        # flat is also changed!
                      [1,2,3,444444,5,6,7,8,9,11,12,13]
5
6 # To avoid this, use .copy()
7 b = flat[3:7].copy() # independent copy
8 b[0] = 22222222
9 print(flat)         # flat is UNCHANGED
```

△ Watch Out!

Slicing = View of the same memory. If you need an independent copy, always use `.copy()`.

3.3 Fancy Indexing

Pass a list of indices to select specific elements:

```
1 arr1 = np.array([10, 20, 30, 40, 50])
2 print(arr1[[0, 1, 2]]) # [10, 20, 30] -- elements at
                        indices 0,1,2
3
4 # Extract even numbers from [1,2,3,4,5,6]
5 my_arr = np.array([1, 2, 3, 4, 5, 6])
6 idx = [1, 3, 5]      # indices of even numbers
7 print(my_arr[idx])    # [2, 4, 6]
```

3.4 Boolean Masking

```
1 arr2 = np.array([10, 12, 13, 344, 43])
2 mask = arr2 > 12      # condition creates a boolean array
3 print(arr2[mask])     # [13, 344, 43] -- only elements
                        where True
4
5 # One-liner:
6 print(arr2[arr2 > 12]) # same result
```

3.5 Random Array Generation

```

1 np.random.rand(3, 3)           # floats in [0.0, 1.0) --
    shape (3,3)
2 np.random.randint(1, 100, (3,3)) # integers in [1,100) --
    shape (3,3)
3 np.random.random((3, 3))       # same as rand (both 0 to 1)
4 np.random.uniform(10, 30, (3,3)) # uniform floats in [10,
    30)
5 np.random.randn(3, 3)          # standard normal
    distribution (mean=0, std=1)

```

Function	Range	Use when
<code>rand()</code>	[0,1) floats	Most common, simple random data
<code>randint()</code>	[low,high) ints	Simulating discrete values
<code>uniform()</code>	[low,high) floats	Random in a custom range
<code>randn()</code>	Normal dist.	ML/statistics (Gaussian noise)

4 Multidimensional Indexing & Axis

4.1 2D Indexing

```

1 arr = np.array([[1, 2, 3],
2                 [4, 5, 6],
3                 [7, 8, 9]])
4
5 arr[0]           # First row: [1, 2, 3]
6 arr[0, 1]        # Row 0, Col 1: 2 (same as arr[0][1])
7 arr[0:2, 1:3]    # Rows 0-1, Cols 1-2: [[2,3],[5,6]]
8
9 # Modify entire column:
10 arr[:, 1] = 0    # Set all elements of column 1 to 0

```

4.2 Axis Concept

```

1 arr = np.array([[1, 2, 3],
2                 [4, 5, 6],
3                 [7, 8, 9]])
4
5 arr.sum(axis=0)   # Sum DOWN the rows (column-wise): [12,
    15, 18]
6 arr.sum(axis=1)   # Sum ACROSS columns (row-wise): [ 6,
    15, 24]

```



```

7
8 # Same works for mean, max, min, etc.
9 arr.mean(axis=0) # column means
10 arr.max(axis=1) # row maximums

```

★ Note

Axis Rule of thumb:

- `axis=0` → work *column-wise* (collapse rows, result has shape of columns)
- `axis=1` → work *row-wise* (collapse columns, result has shape of rows)

4.3 3D Array Indexing

```

1 # Think of 3D as a stack of 2D sheets
2 arr3D = np.array([[[1, 2, 3], [4, 5, 6]],
3                  [[7, 8, 9], [10, 11, 12]]])
4
5 arr3D.shape # (2, 2, 3) -> (depth, rows, cols)
6 arr3D[0, 1, 2] # Sheet 0, Row 1, Col 2 -> 6
7 arr3D[:, 0, :] # All sheets, row 0, all cols:
8                [[1, 2, 3], [7, 8, 9]]
9 arr3D[:, :, 0] # All sheets, all rows, col 0:
10               [[1, 4], [7, 10]]

```

5 Broadcasting

Broadcasting lets NumPy perform operations on arrays of *different shapes* by automatically stretching the smaller array.

5.1 Scalar Broadcasting

```

1 arr = np.array([1, 2, 3, 4, 5])
2 result = arr + 10 # 10 is broadcast to all elements
3 print(result) # [11, 12, 13, 14, 15]
4
5 result = arr ** 2 # Vectorized squaring
6 print(result) # [ 1,  4,  9, 16, 25]

```

5.2 Array Broadcasting

```
1 arr1 = np.array([1, 2, 3])
2 arr2 = np.array([10, 20, 30])
3 print(arr1 + arr2)           # [11, 22, 33]  -- element-wise,
                               same shape
4
5 # 2D + 1D broadcasting:
6 arr1 = np.array([[1, 2, 3],      # shape (2, 3)
7                  [4, 5, 6]])
8 arr2 = np.array([1, 2, 3])        # shape (3,) -- broadcast
                               across rows
9
10 print(arr1 + arr2)
11 # [[2, 4, 6],
12 #   [5, 7, 9]]
```

5.3 Real-World: Data Normalization

```
1 # Dataset: 5 samples, 3 features
2 data = np.array([[10, 20, 30],
3                  [15, 25, 35],
4                  [20, 30, 40],
5                  [25, 35, 45],
6                  [30, 40, 50]])
7
8 mean = data.mean(axis=0)          # Column-wise mean: [20,
9                                30, 40]
10 std  = data.std(axis=0)          # Column-wise std
11 normalized_data = (data - mean) / std # Broadcasting
                                handles it all!
```

✓ Tip

Broadcasting Rules: Arrays are compatible if, for each dimension (right-aligned), the sizes are either equal or one of them is 1.

6 NumPy Data Types

6.1 Integer & Float Types

```

1 arr = np.array([23, 4, 3, 23])
2 arr.dtype                                # int64 (default on 64-bit
    systems)
3
4 arr.astype("float64")                    # Convert: [23., 4., 3.,
    23.]
5
6 arr = np.array([23, 4, 3, 23], dtype="float32") # Set at
    creation
7
8 # Memory impact:
9 arr_int64 = np.array([1, 2, 3], dtype=np.int64)
10 arr_int32 = np.array([1, 2, 3], dtype=np.int32)
11 print(arr_int64.nbytes)                 # 24 bytes (3 * 8 bytes)
12 print(arr_int32.nbytes)                 # 12 bytes (3 * 4 bytes)

```

6.2 Other Data Types

```

1 # String (Unicode) -- avoid in practice
2 arr = np.array(['apple', 'banana', 'cherry'], dtype='U10')
3
4 # Complex numbers
5 arr = np.array([1+2j, 3+4j, 5+6j], dtype='complex128')
6 # dtype='complex128': 64-bit real + 64-bit imaginary
7
8 # Object type (mixed) -- avoid, very slow
9 arr = np.array([{'a': 1}, [1,2,3], 'hello'], dtype=object)

```

6.3 Data Type Quick Reference

dtype	Bytes	Range / Notes	Use case
<code>int8</code>	1	−128 to 127	Tiny integers
<code>int16</code>	2	−32768 to 32767	Small integers
<code>int32</code>	4	$\pm 2.1 \times 10^9$	General integers
<code>int64</code>	8	$\pm 9.2 \times 10^{18}$	Default int
<code>float32</code>	4	~ 7 decimal places	ML weights
<code>float64</code>	8	~ 15 decimal places	Default float
<code>complex128</code>	16	64-bit real + 64-bit imag	Signal processing
<code>bool</code>	1	True / False	Masks, flags

7 Built-in Methods — Quick Reference

```

1 arr = np.array([3, 1, 4, 1, 5, 9, 2, 6])
2
3 np.mean(arr)           # Average value
4 np.std(arr)            # Standard deviation
5 np.var(arr)            # Variance = std^2
6 np.min(arr)            # Minimum value
7 np.max(arr)            # Maximum value
8 np.sum(arr)            # Sum of all elements
9 np.prod(arr)           # Product of all elements
10 np.median(arr)        # Median value
11
12 np.percentile(arr, 50) # 50th percentile (= median)
13 np.percentile(arr, 75) # 75th percentile (Q3)
14
15 np.argmin(arr)         # INDEX of the minimum value
16 np.argmax(arr)        # INDEX of the maximum value
17
18 np.unique(arr)         # Sorted unique elements
19 np.diff(arr)           # Differences between consecutive
    elements
20 np.cumsum(arr)         # Cumulative sum: [3, 4, 8, 9, 14,
    23, 25, 31]
21
22 np.log(arr)            # Natural log (ln)
23 np.exp(arr)            # e^x for each element
24
25 np.corrcoef(arr1, arr2) # Correlation matrix between two
    arrays
26 np.linspace(0, 10, 5)  # 5 values from 0 to 10 evenly
    spaced

```

8 Important NumPy for Data Science (Extra)

These topics weren't in your notebooks but are essential for Data Science / ML work.

8.1 Stacking & Concatenation

```

1 a = np.array([[1, 2], [3, 4]])
2 b = np.array([[5, 6], [7, 8]])
3
4 np.vstack([a, b])      # Vertical stack (add rows)
5 # [[1,2],[3,4],[5,6],[7,8]]
6
7 np.hstack([a, b])      # Horizontal stack (add columns)
8 # [[1,2,5,6],[3,4,7,8]]

```

```

9
10 np.concatenate([a, b], axis=0)    # Same as vstack
11 np.concatenate([a, b], axis=1)    # Same as hstack

```

8.2 Sorting

```

1 arr = np.array([3, 1, 4, 1, 5, 9])
2 np.sort(arr)           # Returns sorted copy: [1, 1, 3, 4,
                        5, 9]
3 np.argsort(arr)        # Returns INDICES that would sort
                        the array
4                        # [1, 3, 0, 2, 4, 5]
5 arr.sort()             # Sorts in-place (modifies original)

```

✓ Tip

`argsort()` is heavily used in ML — e.g., finding the K nearest neighbors by sorting distances.

8.3 Linear Algebra (numpy.linalg)

```

1 A = np.array([[1, 2], [3, 4]])
2 B = np.array([[5, 6], [7, 8]])
3
4 np.dot(A, B)           # Matrix multiplication (dot product)
5 A @ B                  # Same with @ operator (Python 3.5+)
6
7 A.T                    # Transpose: rows become columns
8
9 np.linalg.det(A)        # Determinant
10 np.linalg.inv(A)        # Inverse of matrix
11 np.linalg.eig(A)        # Eigenvalues and eigenvectors
12 np.linalg.norm(A)       # Frobenius norm (matrix magnitude)

```

★ Note

Linear algebra is the backbone of ML. Dot products are used in neural networks; eigenvectors are used in PCA (dimensionality reduction).

8.4 where() — Conditional Selection

```
1 arr = np.array([10, 20, 30, 40, 50])
2
3 np.where(arr > 25, arr, 0)
4 # [0, 0, 30, 40, 50] -- keep value if > 25, else replace
   with 0
5
6 np.where(arr > 25, 'High', 'Low')
7 # ['Low', 'Low', 'High', 'High', 'High']
```

8.5 clip() — Bounding Values

```
1 arr = np.array([-5, 0, 3, 8, 15, 20])
2 np.clip(arr, 0, 10) # All values clamped to [0, 10]
3 # [0, 0, 3, 8, 10, 10]
```

✓ Tip

`clip()` is used in loss functions and gradient clipping in neural networks.

8.6 np.nan and Handling Missing Data

```
1 arr = np.array([1.0, 2.0, np.nan, 4.0, np.nan])
2
3 np.isnan(arr) # [False, False, True, False,
   True]
4 arr[~np.isnan(arr)] # [1., 2., 4.] -- filter out NaNs
5
6 np.nanmean(arr) # Mean ignoring NaN: 2.333...
7 np.nansum(arr) # Sum ignoring NaN: 7.0
8 np.nanmax(arr) # Max ignoring NaN: 4.0
```

△ Watch Out!

Regular `np.mean()` returns `nan` if any `nan` is present. Always use `np.nanmean()`, `np.nansum()`, etc. when your data may have missing values.

8.7 np.random with Seeds (Reproducibility)

```
1 np.random.seed(42)           # Old API: global seed
2 rng = np.random.default_rng(42) # New API: local RNG
   (preferred)
3
4 rng.integers(0, 100, size=(3, 3)) # randint equivalent
5 rng.standard_normal((3, 3))      # randn equivalent
6 rng.random((3, 3))               # rand equivalent
7
8 # Shuffling
9 arr = np.array([1, 2, 3, 4, 5])
10 np.random.shuffle(arr)          # in-place shuffle
11 np.random.choice(arr, size=3, replace=False) # random
   sample
```

★ Note

Always set a seed in ML experiments for **reproducibility** — so your random train/test split gives the same result every time.

8.8 Splitting Arrays

```
1 arr = np.arange(12)
2 np.split(arr, 3)           # Split into 3 equal parts:
   [0-3], [4-7], [8-11]
3 np.array_split(arr, 5)     # Allows unequal splits
4
5 # For 2D arrays:
6 mat = np.arange(16).reshape(4, 4)
7 np.vsplit(mat, 2)          # Split into top-half and
   bottom-half
8 np.hsplit(mat, 2)          # Split into left-half and
   right-half
```

8.9 Useful Utility Functions

```
1 np.abs(arr)                # Absolute value (element-wise)
2 np.sqrt(arr)               # Square root
3 np.floor(arr)               # Round down to nearest int
4 np.ceil(arr)                # Round up to nearest int
5 np.round(arr, 2)            # Round to 2 decimal places
6
7 np.zeros_like(arr)          # Array of 0s with same shape as arr
8 np.ones_like(arr)           # Array of 1s with same shape as arr
```

```

9
10 np.count_nonzero(arr) # Count elements that are not
    zero/False
11 np.any(arr > 5)        # True if ANY element satisfies
    condition
12 np.all(arr > 0)        # True if ALL elements satisfy
    condition

```

8.10 Vectorized Math on Arrays

```

1 arr = np.array([0, np.pi/2, np.pi])
2
3 np.sin(arr)            # [0., 1., 1.2e-16] (approx 0)
4 np.cos(arr)            # [1., 0., -1.]
5 np.tan(arr)
6
7 np.log(arr + 1)        # Natural log (add 1 to avoid
    log(0))
8 np.log2(arr + 1)       # Log base 2
9 np.log10(arr + 1)      # Log base 10
10
11 np.power(2, arr)       # 2^x for each element
12 np.sign(arr)           # -1, 0, or +1 for each element

```

Quick Cheatsheet

NumPy One-Liner Reference

<code>np.array([1,2,3])</code>	Create 1D array
<code>np.zeros((r,c))</code>	$r \times c$ of zeros
<code>np.ones((r,c))</code>	$r \times c$ of ones
<code>np.arange(s,e,step)</code>	Range array
<code>np.linspace(s,e,n)</code>	n evenly spaced
<code>arr.reshape((r,c))</code>	Change shape
<code>arr.flatten()</code>	To 1D
<code>arr[r,c]</code>	2D element access
<code>arr[r1:r2, c1:c2]</code>	2D slicing
<code>arr[mask]</code>	Boolean masking
<code>arr.copy()</code>	Independent copy
<code>np.where(cond,a,b)</code>	Conditional select
<code>np.vstack/hstack</code>	Stack arrays
<code>np.sort / np.argsort</code>	Sort / sort indices
<code>A @ B</code>	Matrix multiply
<code>A.T</code>	Transpose
<code>np.nanmean(arr)</code>	Mean ignoring NaN
<code>np.random.seed(n)</code>	Reproducibility